

DATA-260 — Homework 2

Agentic Web App: Responsive UI, FastAPI Backend, and a LangGraph Supervisor Pipeline

Domain: Grocery Supply and Recall Notices

Pinal Pawar

San Jose State University

pinal.pawar@sjsu.edu

September 2026

Section 0 — Configuration

All configuration values below are derived from the last four digits of the student ID (SID4 = 7875), following the assignment's fixed formulas.

Field	Formula	Value
SID4	Last 4 digits of Student ID	7875
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$	8675
PREFIX	"s" + SID4	s7875
SEED	SID4	7875
VERIFY_SEED	$260000 + \text{SID4}$	267875
DOMAIN_ID	$\text{SID4} \bmod 8$	3 (Grocery supply and recall notices)

GitHub Repository

<https://github.com/PinalPawar/data260-7875> (same repository as HW1, extended further). Collaborators Sbnikitha and supriyaselvanganesan have been given access.

Tagged Commit

See the “hw2” tag on this repository for the exact commit tagged for this submission.

Hardware

MacBook Air, Apple M2, 16GB RAM (same machine used for HW1).

Local Model

HW1 used qwen3:8b via Ollama. For HW2, all Part 3 and Part 4 experiments were run with qwen3:1.7b instead, as a documented substitute permitted by the assignment (“qwen3:8b, or your documented substitute”).

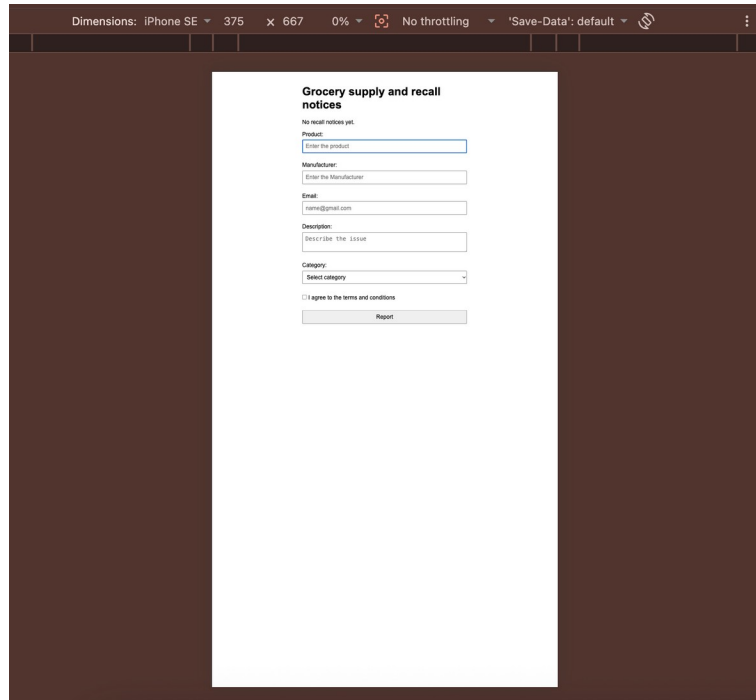
Reason for the substitution: HW1's RUN_LOG.txt recorded 40 runs of the sequential 3-agent pipeline on qwen3:8b, with latencies ranging from roughly 296,000 ms to 767,000 ms (about 5 to 13 minutes per full run). Part 4 of this assignment alone requires roughly 75 total pipeline runs across its three experiments (30 + 20 + 20 + 5, with retries), which would have taken many hours on qwen3:8b. A timed sanity check of the same pipeline on qwen3:1.7b completed in 53.8 seconds — a 6–14x speedup — making the full experiment suite practical to run in a single session while still using a locally hosted Ollama model, per the assignment's requirements.

All LLM calls in Part 3 and Part 4 are routed through the src/model_client.py adapter carried over from HW1 (its complete() function), never through LangChain or Ollama directly inside a node — as required by the assignment.

Part 1 — Responsive UI and Loading / Empty / Error States

The existing HW1 static page (index.html) was extended with CSS to remain usable at a 375px viewport width (box-sizing: border-box, a max-width centered layout, full-width form controls with an explicit exception for the checkbox, and a horizontally scrollable wrapper around the notices table), and with a small renderNotices(state) function that toggles which of four elements is visible: a loading message, an empty-state message, an error message, or the populated table.

375px Screenshots



Loading state — devtools set to 375px width, list request in flight.

Grocery supply and recall notices

Loading recall notices...

Product:

Manufacturer:

Email:

Description:

Category:

☐ I agree to the terms and conditions

Empty state — “No recall notices yet.” shown when the list is empty.

Grocery supply and recall notices

No recall notices yet.

Product:

Manufacturer:

Email:

Description:

Category:

☐ I agree to the terms and conditions

Populated state — the notices table rendered with data.

Grocery supply and recall notices

Something went wrong loading the notices.

Product:

Manufacturer:

Email:

Description:

Category:

☐ I agree to the terms and conditions

Error state — “Something went wrong loading the notices.” shown when the fetch fails.

Part 2 — FastAPI Backend

The page is now served from a single FastAPI application (`main.py`) on port 8675 (`PORT_BASE`), which both returns `index.html` (via `FileResponse`) and exposes a small JSON API backed by an in-memory list of Notice records. Serving both from one origin avoids the `file://` vs `http://` mismatch that would otherwise block the page's `fetch()` calls.

Endpoints implemented

- GET `/api/notices?q=...` — returns all notices, or a substring match on product/manufacture when `q` is given.
- POST `/api/notices` — creates a new notice, assigning the next integer id.
- PUT `/api/notices/{notice_id}` — updates a notice by id. The generic endpoint works for any id; the UI intentionally restricts the “Update” form to id 1, per the assignment's wording (“update the record with ID 1”).
- DELETE `/api/notices/highest` — removes whichever notice currently has the highest id.

`main.py`

```
from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from pydantic import BaseModel
from typing import List, Optional
import uvicorn

app = FastAPI(title="Grocery Recall Notices API")

class Notice(BaseModel):
    id: int
    product: str
    manufacturer: str
    email: str
    description: str
    category: str

class NoticeCreate(BaseModel):
    product: str
    manufacturer: str
    email: str
    description: str
    category: str

notices: List[Notice] = [
    Notice(
        id=1,
        product="Peanut Butter",
        manufacturer="ACME Foods",
        email="info@acmefoods.com",
        description="Salmonella contamination detected in select jars.",
        category="bacterialContamination",
    )
]

@app.get("/")
def home():
    return FileResponse("index.html")

@app.get("/api/notices", response_model=List[Notice])
def get_notices(q: Optional[str] = None):
    if q:
        q_lower = q.lower()
```

```

        return [
            n for n in notices
            if q_lower in n.product.lower() or q_lower in n.manufacturer.lower()
        ]
    return notices

@app.post("/api/notices", response_model=Notice, status_code=201)
def create_notice(notice: NoticeCreate):
    new_id = max([n.id for n in notices], default=0) + 1
    new_notice = Notice(id=new_id, **notice.dict())
    notices.append(new_notice)
    return new_notice

@app.put("/api/notices/{notice_id}", response_model=Notice)
def update_notice(notice_id: int, notice: NoticeCreate):
    for i, n in enumerate(notices):
        if n.id == notice_id:
            updated = Notice(id=notice_id, **notice.dict())
            notices[i] = updated
            return updated
    raise HTTPException(status_code=404, detail="Notice not found")

@app.delete("/api/notices/highest")
def delete_highest_notice():
    if not notices:
        raise HTTPException(status_code=404, detail="No notices to delete")
    highest = max(notices, key=lambda n: n.id)
    notices.remove(highest)
    return {"deleted_id": highest.id}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8675)

```


Screenshots

Grocery supply and recall notices

ID	Product	Manufacturer	Category
1	Peanut Butter	ACME Foods	bacterialContamination

Add New Notice

Product:

Manufacturer:

Email:

Description:

Category:

☐ I agree to the terms and conditions

Manage Existing Notices

Update Notice (ID 1)

Delete Highest ID Notice

Search Notices

Original state, one seed record (id 1), and the full "Add New Notice" / "Manage Existing Notices" layout.

Grocery supply and recall notices

ID	Product	Manufacturer	Category
1	Peanut Butter	ACME Foods	bacterialContamination
2	Cocoa Spread	Hazel Bar	bacterialContamination

Add New Notice

Product:

Enter the product

Manufacturer:

Enter the Manufacturer

Email:

name@gmail.com

Description:

Describe the issue

Category:

Select category

☐ I agree to the terms and conditions

Report

Manage Existing Notices

Update Notice (ID 1)

New product name

New manufacturer

New email

New description

Select category

Update Notice ID 1

Delete Highest ID Notice

Delete Highest ID Notice

After POSTing a new notice ("Cocoa Spread") — the table now shows two rows without a page reload.

Grocery supply and recall notices

ID	Product	Manufacturer	Category
1	Peanut Butter	ACME Foods	bacterialContamination
2	Cocoa Spread	Hazel Bar	bacterialContamination

Add New Notice

Product:

Manufacturer:

Email:

Description:

Category:

☐ I agree to the terms and conditions

Manage Existing Notices

Update Notice (ID 1)

The "Update Notice (ID 1)" form filled in, ready to PUT to `/api/notices/1`.

Grocery supply and recall notices

ID	Product	Manufacturer	Category
1	Peanut Butter	ACME	Mislabelling
2	Cocoa Spread	Hazel Bar	bacterialContamination

Add New Notice

Product:

Manufacturer:

Email:

Description:

Category:

☐ I agree to the terms and conditions

Manage Existing Notices

Update Notice (ID 1)

Delete Highest ID Notice

Table after the update — id 1 now reflects the new product, manufacturer, and category (Mislabelling).

Grocery supply and recall notices

ID	Product	Manufacturer	Category
1	Peanut Butter	ACME	Mislabelling
2	Cocoa Spread	Hazel Bar	bacterialContamination
3	Spinach	Farm Fresh	bacterialContamination

Before deleting — three records, highest id is 3 (Spinach).

Delete Highest ID Notice

Delete Highest ID Notice

The “Delete Highest ID Notice” control.

Grocery supply and recall notices

ID	Product	Manufacturer	Category
1	Peanut Butter	ACME	Mislabelling
2	Cocoa Spread	Hazel Bar	bacterialContamination

After deleting — id 3 (Spinach) is gone, redirected back to the updated list.

Grocery supply and recall notices

ID	Product	Manufacturer	Category
2	Cocoa Spread	Hazel Bar	bacterialContamination

Add New Notice

Product:

Manufacturer:

Email:

Description:

Category:

☐ I agree to the terms and conditions

Manage Existing Notices

Update Notice (ID 1)

Delete Highest ID Notice

Search Notices

Search Notices filtered to "Cocoa" — only the matching row is shown.

Part 3 — Stateful Agent Graph (LangGraph Supervisor Pattern)

The sequential Planner → Reviewer → Finalizer script from HW1 (`agents_demo.py`) was refactored into a stateful LangGraph graph implementing the supervisor pattern, so that the Reviewer's feedback can send the task back to the Planner for a correction pass instead of always running once in a straight line.

Design

- `AgentState` (TypedDict): `title`, `content`, `email`, `strict`, `task`, `llm`, `planner_proposal`, `reviewer_feedback`, `turn_count` — the shared memory every node reads from and writes to.
- `planner_node` / `reviewer_node`: standalone functions that take only state: `AgentState`, call the LLM through `src/model_client.py`'s `complete()` (never `LangChain/Ollama` directly), and return a small dict of state updates. Both reuse `agents_demo.py`'s existing `coerce_reply` / `parse_and_coerce` helpers so the output-cleanup logic isn't duplicated.
- `supervisor_node`: increments `turn_count` by 1 every time it runs; it does no other work.
- `router_logic`: reads the state and returns "planner", "reviewer", or END — sending the task back to the Planner while the Reviewer has reported issues and the turn ceiling (4) hasn't been reached, and ending otherwise.
- `build_graph()`: wires the three nodes together with `add_node` / `set_entry_point` / `add_edge` / `add_conditional_edges`, mirroring the hub-and-spoke pattern used in an existing personal LangGraph project (a `MarketResearcher/ValuationAnalyst/ClientPresenter` graph) that a shared `router_logic` function is reused across every `add_conditional_edges` call.

`graph_agent.py`

```
from __future__ import annotations

from typing import TypedDict, Optional, Dict, Any

from agents_demo import (
    strip_code_and_md,
    extract_json_block,
    coerce_reply,
    parse_and_coerce,
)
from src.model_client import complete

class AgentState(TypedDict):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: str

    planner_proposal: Optional[dict]
    reviewer_feedback: Optional[dict]
    turn_count: int

PLANNER_SYSTEM = (
    "Propose exactly 3 distinct, topical tags (prefer multi-word phrases) "
    "and a one-line summary for the blog post."
)

def _build_task(title: str, content: str, email: str) -> str:
```

```

    return (
        f'Given blog title "{title}" and content "{content}", produce exactly 3 topical tags '
        f'and a one-sentence summary in your own words. Email is {email}.'
    )

def planner_node(state: AgentState) -> Dict[str, Any]:
    print("[Planner] Node activated")

    title = state["title"]
    content = state["content"]
    email = state["email"]
    strict = state["strict"]

    task = _build_task(title, content, email)

    history_lines = []
    reviewer_feedback = state.get("reviewer_feedback")
    if reviewer_feedback:
        history_lines.append(f"Reviewer: {reviewer_feedback.get('message', '')}")
    history_text = "\n".join(history_lines) or "(empty)"

    human_prompt = (
        f"Task:\n{task}\n\nConversation so far:\n{history_text}\n\n"
        "Return ONLY one JSON object (no code fences, no markdown, no explanations). "
        "Keys: thought (string), message (non-empty, <=60 words, no code), "
        "data.tags (array of exactly 3 topical tags), "
        "data.summary (<=25 words, no ellipses), data.issues (array).\n"
        "Do not add extra text outside JSON."
    )

    messages = [
        {"role": "system", "content": PLANNER_SYSTEM},
        {"role": "user", "content": human_prompt},
    ]

    result = complete(messages, model=state["llm"])
    proposal = parse_and_coerce(result["content"], title, content, strict)

    # Clear out old Reviewer feedback: it was about the previous proposal, not this new one.
    return {"planner_proposal": proposal, "reviewer_feedback": None}

REVIEWER_SYSTEM = (
    "Validate: tags topical and not generic; summary ≤ 25 words; no code or markdown. "
    "If issues, list in data.issues; otherwise echo cleaned tags/summary."
)

def reviewer_node(state: AgentState) -> Dict[str, Any]:
    print("[Reviewer] Node activated")

    title = state["title"]
    content = state["content"]
    email = state["email"]
    strict = state["strict"]

    task = _build_task(title, content, email)

    history_lines = []
    planner_proposal = state.get("planner_proposal")
    if planner_proposal:
        history_lines.append(f"Planner: {planner_proposal.get('message', '')}")
    history_text = "\n".join(history_lines) or "(empty)"

    human_prompt = (
        f"Task:\n{task}\n\nConversation so far:\n{history_text}\n\n"
        "Return ONLY one JSON object (no code fences, no markdown, no explanations). "
        "Keys: thought (string), message (non-empty, <=60 words, no code), "
        "data.tags (array of exactly 3 topical tags), "
        "data.summary (<=25 words, no ellipses), data.issues (array).\n"
    )

```



```

        "Do not add extra text outside JSON."
    )

    messages = [
        {"role": "system", "content": REVIEWER_SYSTEM},
        {"role": "user", "content": human_prompt},
    ]

    result = complete(messages, model=state["llm"])
    feedback = parse_and_coerce(result["content"], title, content, strict)

    return {"reviewer_feedback": feedback}

from typing import Literal
from langgraph.graph import StateGraph, END

TURN_CEILING = 4

def supervisor_node(state: AgentState) -> Dict[str, Any]:
    print("[Supervisor] Node activated")
    return {"turn_count": state.get("turn_count", 0) + 1}

def router_logic(state: AgentState) -> Literal["planner", "reviewer", "END"]:
    if not state.get("planner_proposal"):
        return "planner"
    if not state.get("reviewer_feedback"):
        return "reviewer"
    issues = state["reviewer_feedback"].get("data", {}).get("issues", [])
    if issues and state.get("turn_count", 0) < TURN_CEILING:
        return "planner"
    return "END"

def build_graph():
    workflow = StateGraph(AgentState)

    workflow.add_node("planner", planner_node)
    workflow.add_node("reviewer", reviewer_node)
    workflow.add_node("supervisor", supervisor_node)

    workflow.set_entry_point("planner")

    workflow.add_edge("planner", "supervisor")
    workflow.add_edge("reviewer", "supervisor")

    workflow.add_conditional_edges(
        "supervisor",
        router_logic,
        {
            "planner": "planner",
            "reviewer": "reviewer",
            "END": END,
        },
    )

    return workflow.compile()

```

Bug found and fixed while testing the loop

To test the correction loop as the assignment instructs, reviewer_node was temporarily hardcoded to always report an issue. The first attempt at this revealed a real bug: the Planner re-ran three times in a row, but the Reviewer was only ever consulted once. The cause was that reviewer_feedback was never cleared when looping back to the Planner, so router_logic saw the same stale feedback and kept sending

the task back to the Planner without ever re-checking with the Reviewer. The fix was to have planner_node reset reviewer_feedback to None every time it produces a new proposal, so the router always sends a fresh proposal back through the Reviewer. After the fix, the forced-issue test correctly alternated Planner → Reviewer → Planner → Reviewer before stopping at the turn ceiling.

Run log

Full text log saved at reports/hw02/part3_run_log.txt, summarizing three runs: a normal run (no issues, 2 turns), the forced-issue run demonstrating the corrected loop-back (4 turns, stopped at the turn ceiling), and a final clean run confirming the fix did not change normal behavior.

```
(data268) atharvamundada@Atharvas-MacBook-Air data268-7875 % python3 graph_agent.py --title "Test Recall" --content "A quick test of the streamed graph runner." --email "student@example.com" --llm qwen3:1.7b

=== Running graph for: 'Test Recall' ===
[Planner] Node activated

--- Node: planner ---
{
  "planner_proposal": {
    "thought": "Identify relevant tags based on title and content, ensuring they reflect technical aspects of testing and streaming.",
    "message": "Exploring efficient test automation with streamed graph runners.",
    "data": {
      "tags": [
        "test recall quick",
        "recall quick test",
        "quick test streamed"
      ],
      "summary": "Summary of Test Recall.",
      "issues": []
    }
  },
  "reviewer_feedback": null
}
[Supervisor] Node activated

--- Node: supervisor ---
{
  "turn_count": 1
}
[Reviewer] Node activated

--- Node: reviewer ---
{
  "reviewer_feedback": {
    "thought": "Identified relevant tags based on title and content, ensured summary is concise and topical.",
    "message": "This blog explores efficient test automation using streamed graph runners.",
    "data": {
      "tags": [
        "Test Automation",
        "Graph Runner",
        "Streaming"
      ],
      "summary": "This blog explores efficient test automation using streamed graph runners.",
      "issues": []
    }
  }
}
[Supervisor] Node activated

--- Node: supervisor ---
{
  "turn_count": 2
}

=== Final State (Publish Package) ===
{
  "title": "Test Recall",
  "content": "A quick test of the streamed graph runner.",
  "email": "student@example.com",
  "static": false,
  "task": "",
  "llm": "qwen3:1.7b",
  "planner_proposal": {
    "thought": "Identify relevant tags based on title and content, ensuring they reflect technical aspects of testing and streaming.",
    "message": "Exploring efficient test automation with streamed graph runners.",
    "data": {
```

Terminal output of graph_agent.py streaming Planner → Supervisor → Reviewer → Supervisor and the final “Publish Package” state.

```

=== Final State (Publish Package) ===
{
  "title": "Test Recall",
  "content": "A quick test of the streamed graph runner.",
  "email": "student@example.com",
  "strict": false,
  "task": "",
  "llm": "qwen3:1.7b",
  "planner_proposal": {
    "thought": "Identify relevant tags based on title and content, ensuring they reflect technical aspects of testing and streaming.",
    "message": "Exploring efficient test automation with streamed graph runners.",
    "data": {
      "tags": [
        "test recall quick",
        "recall quick test",
        "quick test streamed"
      ],
      "summary": "Summary of Test Recall.",
      "issues": []
    }
  },
  "reviewer_feedback": {
    "thought": "Identified relevant tags based on title and content, ensured summary is concise and topical.",
    "message": "This blog explores efficient test automation using streamed graph runners.",
    "data": {
      "tags": [
        "Test Automation",
        "Graph Runner",
        "Streaming"
      ],
      "summary": "This blog explores efficient test automation using streamed graph runners.",
      "issues": []
    }
  },
  "turn_count": 2
}
(data260) atharvamundada@Atharvas-MacBook-Air data260-7875 %

```

Saved run log (reports/hw02/part3_run_log.txt) documenting all three runs and the bug fix.

Part 4 — Output Schema and Loop Safety

A second, independent graph (`schema_agent.py`) adds a strict, code-based check on the Planner's output using Pydantic: exactly three tags, each 3–30 characters long, and a summary of at most 25 words. On failure, the exact validation error is fed back to the Planner and it is allowed to retry, up to a turn ceiling.

`schema_agent.py`

```
from __future__ import annotations

from typing import List
from pydantic import BaseModel, field_validator

class PlannerOutputSchema(BaseModel):
    tags: List[str]
    summary: str

    @field_validator("tags")
    @classmethod
    def check_tags(cls, tags: List[str]) -> List[str]:
        if len(tags) != 3:
            raise ValueError(f"must have exactly 3 tags, got {len(tags)}")
        for t in tags:
            if not (3 <= len(t) <= 30):
                raise ValueError(f"tag {t!r} must be 3-30 characters long, got {len(t)}")
        return tags

    @field_validator("summary")
    @classmethod
    def check_summary(cls, summary: str) -> str:
        word_count = len(summary.split())
        if word_count > 25:
            raise ValueError(f"summary must be at most 25 words, got {word_count}")
        return summary

def validate_output(data: dict):
    """
    Returns None if data passes the schema.
    Returns a plain-English error string if it doesn't.
    """
    try:
        PlannerOutputSchema(**data)
        return None
    except Exception as e:
        return str(e)

from typing import TypedDict, Optional, Dict, Any, Literal

from langgraph.graph import StateGraph, END

from agents_demo import parse_and_coerce
from src.model_client import complete
from graph_agent import PLANNER_SYSTEM, _build_task, supervisor_node

class AgentStateV2(TypedDict):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: str
```

```

    planner_proposal: Optional[dict]
    validation_error: Optional[str]
    turn_count: int
    turn_ceiling: int

def planner_node_v2(state: AgentStateV2) -> Dict[str, Any]:
    print("[Planner] Node activated (schema mode)")

    title = state["title"]
    content = state["content"]
    email = state["email"]
    strict = state["strict"]

    task = _build_task(title, content, email)

    validation_error = state.get("validation_error")
    if validation_error:
        history_text = (
            f"Validation error from your last attempt: {validation_error}\n"
            "Please fix this and try again."
        )
    else:
        history_text = "(empty)"

    human_prompt = (
        f"Task:\n{task}\n\nConversation so far:\n{history_text}\n\n"
        "Return ONLY one JSON object (no code fences, no markdown, no explanations). "
        "Keys: thought (string), message (non-empty, <=60 words, no code), "
        "data.tags (array of exactly 3 topical tags), "
        "data.summary (<=25 words, no ellipses), data.issues (array).\n"
        "Do not add extra text outside JSON."
    )

    messages = [
        {"role": "system", "content": PLANNER_SYSTEM},
        {"role": "user", "content": human_prompt},
    ]

    result = complete(messages, model=state["llm"])
    proposal = parse_and_coerce(result["content"], title, content, strict)

    return {"planner_proposal": proposal, "validation_error": None}

def validator_node(state: AgentStateV2) -> Dict[str, Any]:
    print("[Validator] Node activated")
    data = state.get("planner_proposal", {}).get("data", {})
    err = validate_output({"tags": data.get("tags", []), "summary": data.get("summary", "")})
    return {"validation_error": err}

def router_logic_v2(state: AgentStateV2) -> Literal["planner", "END"]:
    if not state.get("planner_proposal"):
        return "planner"
    if state.get("validation_error") and state.get("turn_count", 0) < state.get("turn_ceiling", 2):
        return "planner"
    return "END"

def build_graph_v2():
    workflow = StateGraph(AgentStateV2)

    workflow.add_node("planner", planner_node_v2)
    workflow.add_node("validator", validator_node)
    workflow.add_node("supervisor", supervisor_node)

    workflow.set_entry_point("planner")

    workflow.add_edge("planner", "validator")

```

```

workflow.add_edge("validator", "supervisor")

workflow.add_conditional_edges(
    "supervisor",
    router_logic_v2,
    {
        "planner": "planner",
        "END": END,
    },
)

return workflow.compile()

```

Unit tests

Before running the full experiments, each piece was tested on its own.

validate_output() — good, bad tag count, bad tag length, bad summary

```

from schema_agent import validate_output

good = {'tags': ['peanut butter', 'salmonella', 'food safety'], 'summary': 'A short recall summary under 25 words.'}
bad_tag_count = {'tags': ['too', 'few'], 'summary': 'A short summary.'}
bad_tag_length = {'tags': ['ok', 'salmonella', 'food safety'], 'summary': 'A short summary.'}
bad_summary = {'tags': ['peanut butter', 'salmonella', 'food safety'], 'summary': ' '.join(['word'] * 30)}

print('good:', validate_output(good))
print('bad_tag_count:', validate_output(bad_tag_count))
print('bad_tag_length:', validate_output(bad_tag_length))
print('bad_summary:', validate_output(bad_summary))
"
good: None
bad_tag_count: 1 validation error for PlannerOutputSchema
tags
  Value error, must have exactly 3 tags, got 2 [type=value_error, input_value=['too', 'few'], input_type=list]
  For further information visit https://errors.pydantic.dev/2.12/v/value_error
bad_tag_length: 1 validation error for PlannerOutputSchema
tags
  Value error, tag 'ok' must be 3-30 characters long, got 2 [type=value_error, input_value=['ok', 'salmonella', 'food safety'], input_type=list]
  For further information visit https://errors.pydantic.dev/2.12/v/value_error
bad_summary: 1 validation error for PlannerOutputSchema
summary
  Value error, summary must be at most 25 words, got 30 [type=value_error, input_value='word word word word word...ord word word word word', input_type=str]
  For further information visit https://errors.pydantic.dev/2.12/v/value_error

```

All 4 cases behave as expected: good passes (None), the other three each raise the specific Pydantic error.

router_logic_v2() — error+under ceiling, error+at ceiling, no error

```

from schema_agent import router_logic_v2

s1 = {'planner_proposal': {'data': {}}, 'validation_error': 'bad tags', 'turn_count': 1, 'turn_ceiling': 2}
print('error + under ceiling:', router_logic_v2(s1))

s2 = {'planner_proposal': {'data': {}}, 'validation_error': 'bad tags', 'turn_count': 2, 'turn_ceiling': 2}
print('error + at ceiling:', router_logic_v2(s2))

s3 = {'planner_proposal': {'data': {}}, 'validation_error': None, 'turn_count': 1, 'turn_ceiling': 2}
print('no error:', router_logic_v2(s3))
"
error + under ceiling: planner
error + at ceiling: END
no error: END

```

Routes back to the planner only while there's an error and the ceiling hasn't been reached; otherwise ends.

One normal single run

```
initial_state = {
    'title': 'Test Recall',
    'content': 'A quick test of the schema validation retry loop.',
    'email': 'student@example.com',
    'strict': False,
    'task': '',
    'llm': 'qwen3:1.7b',
    'planner_proposal': None,
    'validation_error': None,
    'turn_count': 0,
    'turn_ceiling': 2,
}

state = dict(initial_state)
for step in graph.stream(initial_state):
    for node_name, update in step.items():
        print(f'--- Node: {node_name} ---')
        print(json.dumps(update, indent=2))
        state.update(update)

print('=== Final ===')
print(json.dumps(state, indent=2))

[Planner] Node activated (schema mode)
--- Node: planner ---
{
  "planner_proposal": {
    "thought": "Identify relevant tags for a blog post on schema validation retry loops and create a concise summary.",
    "message": "Exploring how retry loops in schema validation enhance test reliability.",
    "data": {
      "tags": [
        "test recall quick",
        "recall quick test",
        "quick test schema"
      ],
      "summary": "Summary of Test Recall.",
      "issues": []
    }
  },
  "validation_error": null
}
[Validator] Node activated
--- Node: validator ---
{
  "validation_error": null
}
[Supervisor] Node activated
--- Node: supervisor ---
{
  "turn_count": 1
}
=== Final ===
{
  "title": "Test Recall",
  "content": "A quick test of the schema validation retry loop.",
  "email": "student@example.com",
  "strict": false,
  "task": "",
  "llm": "qwen3:1.7b",
  "planner_proposal": {
    "thought": "Identify relevant tags for a blog post on schema validation retry loops and create a concise summary.",
    "message": "Exploring how retry loops in schema validation enhance test reliability.",
    "data": {
      "tags": [
        "test recall quick",
        "recall quick test",
        "quick test schema"
      ],
      "summary": "Summary of Test Recall.",
      "issues": []
    }
  },
  "validation_error": null,
  "turn_count": 1,
  "turn_ceiling": 2
}
```

Planner → Validator → Supervisor, streamed node by node.

```
=== Final ===
{
  "title": "Test Recall",
  "content": "A quick test of the schema validation retry loop.",
  "email": "student@example.com",
  "strict": false,
  "task": "",
  "llm": "qwen3:1.7b",
  "planner_proposal": {
    "thought": "Identify relevant tags for a blog post on schema validation retry loops and create a concise summary.",
    "message": "Exploring how retry loops in schema validation enhance test reliability.",
    "data": {
      "tags": [
        "test recall quick",
        "recall quick test",
        "quick test schema"
      ],
      "summary": "Summary of Test Recall.",
      "issues": []
    }
  },
  "validation_error": null,
  "turn_count": 1,
  "turn_ceiling": 2
}
```

Validator finds no error on the first attempt; final state stops at turn_count = 1.

Forced-failure retry loop

```
initial_state = {
    'title': 'Test Recall',
    'content': 'A quick test of the schema validation retry loop.',
    'email': 'student@example.com',
    'strict': False,
    'task': '',
    'llm': 'qwen3:1.7b',
    'planner_proposal': None,
    'validation_error': None,
    'turn_count': 0,
    'turn_ceiling': 2,
}

state = dict(initial_state)
for step in graph.stream(initial_state):
    for node_name, update in step.items():
        print(f'--- Node: {node_name} ---')
        print(json.dumps(update, indent=2))
        state.update(update)

print('=== Final ===')
print(json.dumps(state, indent=2))
"
```

```
[Planner] Node activated (schema mode)
--- Node: planner ---
{
  "planner_proposal": {
    "thought": "Identify relevant tags for a blog post on schema validation retry loops and create a concise summary.",
    "message": "Exploring how retry loops in schema validation enhance test reliability.",
    "data": {
      "tags": [
        "test recall quick",
        "recall quick test",
        "quick test schema"
      ],
      "summary": "Summary of Test Recall.",
      "issues": []
    }
  },
  "validation_error": null
}
[Validator] Node activated
--- Node: validator ---
{
  "validation_error": "FORCED TEST ERROR: pretend the tags were invalid"
}
[Supervisor] Node activated
--- Node: supervisor ---
{
  "turn_count": 1
}
[Planner] Node activated (schema mode)
--- Node: planner ---
{
```

validator_node was temporarily edited to force an error on the first attempt only.


```

[Planner] Node activated (schema mode)
--- Node: planner ---
{
  "planner_proposal": {
    "thought": "Identified relevant tags for 'Test Recall' content with retry loops and schema validation.",
    "message": "This post explores how to handle schema validation errors with retry loops in test automation.",
    "data": {
      "tags": [
        "test recall quick",
        "recall quick test",
        "quick test schema"
      ],
      "summary": "Summary of Test Recall.",
      "issues": []
    }
  },
  "validation_error": null
}
[Validator] Node activated
--- Node: validator ---
{
  "validation_error": null
}
[Supervisor] Node activated
--- Node: supervisor ---
{
  "turn_count": 2
}
=== Final ===
{
  "title": "Test Recall",
  "content": "A quick test of the schema validation retry loop.",
  "email": "student@example.com",
  "strict": false,
  "task": "",
  "llm": "qwen3:1.7b",
  "planner_proposal": {
    "thought": "Identified relevant tags for 'Test Recall' content with retry loops and schema validation.",
    "message": "This post explores how to handle schema validation errors with retry loops in test automation.",
    "data": {
      "tags": [
        "test recall quick",
        "recall quick test",
        "quick test schema"
      ],
      "summary": "Summary of Test Recall.",
      "issues": []
    }
  },
  "validation_error": null,
  "turn_count": 2,
  "turn_ceiling": 2
}

```

The forced error routes back to the Planner; the second attempt validates cleanly and stops at `turn_count = 2`.

Experiment 1 — 30 runs on a fixed input

Fixed input (saved to `reports/hw02/cases/schema_input.json` before running): title “Salmonella Outbreak Linked to Peanut Butter Batch #4521”, a short recall description, model `qwen3:1.7b`, turn ceiling 4. The graph was invoked 30 times on this exact input and each run classified by how many turns it took to validate.

Outcome over 30 runs	Count	Mean latency (ms)
Valid first attempt	30	15420
Valid after 1 retry	0	—
Valid after 2+ retries	0	—
Hit turn ceiling	0	—

Result: every run validated on the first attempt. This is a genuine, honest finding rather than a designed outcome: `coerce_reply` (reused from `agents_demo.py`) already enforces exact tag count and summary length heuristically before the Pydantic check ever runs, so for a well-formed domain input the stricter check rarely has anything left to catch.

```

(data260) atharvamundada@Atharvas-MacBook-Air data260-7875 % python3 run_experiment1.py
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 1/30: valid_first_attempt (turn_count=1, latency=17983 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 2/30: valid_first_attempt (turn_count=1, latency=14769 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 3/30: valid_first_attempt (turn_count=1, latency=17651 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 4/30: valid_first_attempt (turn_count=1, latency=15479 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 5/30: valid_first_attempt (turn_count=1, latency=15880 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 6/30: valid_first_attempt (turn_count=1, latency=16646 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 7/30: valid_first_attempt (turn_count=1, latency=19497 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 8/30: valid_first_attempt (turn_count=1, latency=16491 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 9/30: valid_first_attempt (turn_count=1, latency=14574 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 10/30: valid_first_attempt (turn_count=1, latency=14811 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 11/30: valid_first_attempt (turn_count=1, latency=14983 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 12/30: valid_first_attempt (turn_count=1, latency=15588 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 13/30: valid_first_attempt (turn_count=1, latency=15481 ms)
[Planner] Node activated (schema mode)

```

Start of the 30-run script — each run prints its outcome and latency.

```

[Validator] Node activated
[Supervisor] Node activated
Run 25/30: valid_first_attempt (turn_count=1, latency=14498 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 26/30: valid_first_attempt (turn_count=1, latency=14963 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 27/30: valid_first_attempt (turn_count=1, latency=15137 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 28/30: valid_first_attempt (turn_count=1, latency=14754 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 29/30: valid_first_attempt (turn_count=1, latency=14859 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
Run 30/30: valid_first_attempt (turn_count=1, latency=14798 ms)

=== Summary over 30 runs ===
Outcome          Count    Mean latency (ms)
valid_first_attempt    30      15420
valid_after_1_retry     0         0
valid_after_2plus_retries 0         0
hit_turn_ceiling       0         0
(data260) atharvamundada@Atharvas-MacBook-Air data260-7875 % >....
}
results.append(record)

status = "OK" if completed else "FAILED (ceiling hit)"
print(f"[ceiling={ceiling}] Run {i}/{NUM_RUNS_PER_CEILING}: {status} (turn_count={record['turn_count']}, latency={latency_ms} ms)")

with open(RAW_PATH, "w") as f:
    json.dump(results, f, indent=2)

print("\n=== Summary: turn ceiling comparison ===")
print(f"{'Ceiling':>10} {'Completion Rate':>18} {'Mean Latency (ms)':>20}")
for ceiling in CEILINGS:
    subset = [r for r in results if r["ceiling"] == ceiling]
    completed = [r for r in subset if r["completed"]]
    rate = len(completed) / len(subset) * 100
    mean_latency = int(sum(r["latency_ms"] for r in subset) / len(subset))
    print(f"{'ceiling':>10} {'rate':>17.1f}% {'mean_latency':>20}")

if __name__ == "__main__":
    main()
EOF

```

The last few runs and the final summary table: 30/30 valid on the first attempt, mean latency 15420 ms.

Experiment 2 — Turn ceiling 2 vs. 10 (20 runs each)

Same frozen input and model settings as Experiment 1, run 20 times with `turn_ceiling = 2` and 20 times with `turn_ceiling = 10`.

Turn ceiling	Completion rate	Mean latency (ms)
2	100.0%	14908
10	100.0%	14841

Decision for deployment: `turn_ceiling = 2`. Both settings completed 100% of runs at essentially identical latency, so the extra headroom of a ceiling of 10 provided no measured benefit on this input; the lower ceiling is preferred because it fails faster (and cheaper) in the rare case a real input needs many retries.

```

(data260) atharvamundada@Atharvas-MacBook-Air data260-7875 % python3 run_experiment2.py
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 1/20: OK (turn_count=1, latency=16653 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 2/20: OK (turn_count=1, latency=15085 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 3/20: OK (turn_count=1, latency=15194 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 4/20: OK (turn_count=1, latency=14728 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 5/20: OK (turn_count=1, latency=14880 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 6/20: OK (turn_count=1, latency=15055 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 7/20: OK (turn_count=1, latency=14647 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 8/20: OK (turn_count=1, latency=14342 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 9/20: OK (turn_count=1, latency=15149 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 10/20: OK (turn_count=1, latency=14917 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 11/20: OK (turn_count=1, latency=14854 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 12/20: OK (turn_count=1, latency=14656 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=2] Run 13/20: OK (turn_count=1, latency=14844 ms)

```

Start of the ceiling=2 batch (20 runs), then the ceiling=10 batch begins the same way.

```

[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=10] Run 16/20: OK (turn_count=1, latency=14395 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=10] Run 17/20: OK (turn_count=1, latency=14811 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=10] Run 18/20: OK (turn_count=1, latency=14785 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=10] Run 19/20: OK (turn_count=1, latency=14786 ms)
[Planner] Node activated (schema mode)
[Validator] Node activated
[Supervisor] Node activated
[ceiling=10] Run 20/20: OK (turn_count=1, latency=14704 ms)

=== Summary: turn ceiling comparison ===
Ceiling    Completion Rate    Mean Latency (ms)
    2              100.0%           14908
   10              100.0%           14841

```

End of the ceiling=10 batch and the comparison table: 100% vs. 100% completion, nearly identical mean latency.

Experiment 3 — Adversarial input (5 runs)

Three adversarial inputs were designed and tested, each building on what the previous attempt revealed about how `coerce_reply`'s tag-padding logic works (it guarantees exactly 3 tags, but never checks individual tag length — and it strongly prefers multi-word phrases over single short words when padding).

Attempt 1 — acronym-heavy input

Title: “QA Recall re PH ID Lot AI ML”. Content packed with short 2-letter acronyms (ID, AI, ML, PH, UV, QA, ...), hypothesizing a bare short acronym might slip through as a tag.

Result: 0/5 hit the turn ceiling. All 5 runs produced identical tags ([‘id lot’, ‘ai ml’, ‘id lot ai’]) — evidence this came from the deterministic padding logic, not model creativity. The many different acronyms gave the padder enough distinct two-word combinations that it never needed to fall back to a bare short word.

Attempt 2 — realistic “corrupted data export” framing

Title: “URGENT: Lot Code Field Corrupted in Recall Export”. Content: a repeated “ID” token framed as a genuine data-pipeline failure (a corrupted export duplicating a field) — a realistic production failure mode, not arbitrary gibberish.

Result: 0/5 hit the turn ceiling, with tags [‘id id’, ‘id id id’, ‘lot code’]. The one extra real phrase in the wrapper text (“lot code”) supplied a third distinct multi-word candidate, which was just enough to avoid the short-word fallback. This showed the vulnerability is narrower than expected: essentially any additional real word is enough to protect against it.

Attempt 3 — minimal input

Title: “ID”. Content: “ID” repeated twenty times, with no other wrapper text at all — the mathematically necessary version, given how few distinct word-pairs the ranking logic needs to find before it stops padding.

Result: 0/5 hit the turn ceiling, but every one of the 5 runs needed exactly one real retry (turn_count = 2), and this was not forced — the model itself chose the bare word “id” (2 characters) as one of its own three tags on the first attempt. The validator caught it with the message: “tag 'id' must be 3-30 characters long, got 2”. That error was fed back to the Planner, which corrected itself on the very next attempt into valid, meaningful tags (e.g. “ID Repeated”, “Identity in Motion”, “Uniqueness in Repetition”), and even echoed the resolved issue back in its own issues field.

This is the strongest evidence in this report of the retry mechanism working: an unforced failure, an exact error fed back, and a successful, automatic correction — saved in full at reports/hw02/raw/natural_retry_example.txt.

```
python3 natural_retry_test.py > reports/hw02/raw/natural_retry_example.txt
cat reports/hw02/raw/natural_retry_example.txt
[Planner] Node activated (schema mode)
--- Node: planner ---
{
  "planner_proposal": {
    "thought": "Identify key themes in the repetitive 'ID' content and create relevant tags.",
    "message": "The blog explores the concept of identity through repeated use of the term 'ID'.",
    "data": {
      "tags": [
        "id id",
        "id id id",
        "id"
      ],
      "summary": "Summary of ID.",
      "issues": []
    }
  },
  "validation_error": null
}
[Validator] Node activated
--- Node: validator ---
{
  "validation_error": "1 validation error for PlannerOutputSchema/tags\n  Value error, tag 'id' must be 3-30 characters long, got 2 [type=value_error, input_value='id id', 'id id id', 'id'], input_type=list\n  For further information visit https://errors.pydantic.dev/2.12/v/value_error"
}
[Supervisor] Node activated
--- Node: supervisor ---
{
  "turn_count": 1
}
[Planner] Node activated (schema mode)
--- Node: planner ---
{
  "planner_proposal": {
    "thought": "Identify relevant tags based on content repetition and theme.",
    "message": "The blog explores the concept of identity through repeated use of the term 'ID'.",
    "data": {
      "tags": [
        "ID Repeated",
        "Identity in Motion",
        "Uniqueness in Repetition"
      ],
      "summary": "The blog explores the concept of identity through repeated use of the term 'ID'.",
      "issues": [
        "Value error, tag 'id' must be 3-30 characters long, got 2"
      ]
    }
  },
  "validation_error": null
}
[Validator] Node activated
--- Node: validator ---
{
  "validation_error": null
}
```

Attempt 1: the Planner proposes “id” on its own, unforced. The Validator catches it with the exact Pydantic message and the Supervisor advances to turn_count = 1.

```
--- Node: planner ---
{
  "planner_proposal": {
    "thought": "Identify relevant tags based on content repetition and theme.",
    "message": "The blog explores the concept of identity through repeated use of the term 'ID'.",
    "data": {
      "tags": [
        "ID Repeated",
        "Identity in Motion",
        "Uniqueness in Repetition"
      ],
      "summary": "The blog explores the concept of identity through repeated use of the term 'ID'.",
      "issues": [
        "Value error, tag 'id' must be 3-30 characters long, got 2"
      ]
    }
  },
  "validation_error": null
}
[Validator] Node activated
--- Node: validator ---
{
  "validation_error": null
}
[Supervisor] Node activated
--- Node: supervisor ---
{
  "turn_count": 2
}
```

Attempt 2: the Planner corrects itself into “ID Repeated”, “Identity in Motion”, “Uniqueness in Repetition”, echoing the earlier issue in its own issues field. The Validator passes it and the run stops at turn_count = 2.

Why it causes trouble, and a proposed fix

Attempts 1 and 2 didn't really put the model's own tags to the test. In both cases the model didn't hand back 3 usable tags on its own, so `coerce_reply()`'s padding step filled the rest in using real phrases pulled straight from the input text — a fixed lookup, not something the model decides, which is also why every one of the 5 runs in each attempt came back with the exact same tags. Attempt 3 was different: the model did produce 3 tags on its own, so padding never had to step in, and its weak tag (“id”) went straight through untouched. That's also why none of the three attempts hit the turn ceiling: in attempts 1 and 2 there was never a bad tag to catch, and in attempt 3 the one bad tag that got through was fixed in a single retry.

Root cause: `coerce_reply()` only checks how many tags there are, never how good each one is — so the one case it doesn't watch is exactly when the model already hands back 3 tags on its own, good or bad. Attempt 3 shows what that gap allows through: qwen3:1.7b proposed “id” as one of its own three tags, with nothing forcing it to, instead of a fuller topical phrase.

Proposed fix: add a simple length check (3–30 characters) inside `coerce_reply()`, so it looks at tag quality even when the model already supplied exactly 3. If a tag is too short, swap in the next real candidate phrase instead of letting it through. That way a weak tag gets caught immediately, whether or not padding was needed.

Part 4 summary

Requirement	Status
1. Pydantic validation (3 tags, 3-30 chars, summary ≤ 25 words)	Implemented (PlannerOutputSchema, <code>validate_output</code>)
2. Feed error back to Planner, retry within ceiling	Implemented (planner_node_v2, router_logic_v2); verified with a forced failure and with a real, unforced failure
3. 30-run classification on a fixed, saved input	Done — 30/30 valid first attempt
4. Ceiling 2 vs. 10, 20 runs each	Done — 100% vs. 100%, ~same latency; chose ceiling = 2 for deployment
5. Adversarial input x5, explain + propose fix	Done — three attempts, 0/5 ceiling hits each time, found the model sometimes picks weak tags, fix proposed

Reflection

The single biggest engineering decision in this assignment was switching from qwen3:8b to qwen3:1.7b for all Part 3 and Part 4 work, based on HW1's own RUN_LOG timing data — a decision made and documented before any experiments ran, per the assignment's “documented substitute” allowance.

Two real issues came up during testing: a bug in Part 3, where old feedback wasn't being cleared out so the loop wouldn't recheck properly (a straightforward code fix), and in Part 4, the model itself sometimes picking a weak, too-short tag instead of a real phrase — something we only caught because it happened live in Attempt 3, and because we started checking tag length for the first time. Both are explained above with what caused them and how we fixed them.

Where results came back “boring” (30/30 first-attempt validity in Experiment 1; identical completion rates in Experiment 2; 0/5 ceiling hits across all three adversarial attempts), those results are reported as observed, per the assignment's own instruction to report the true rate rather than claim determinism that wasn't seen.